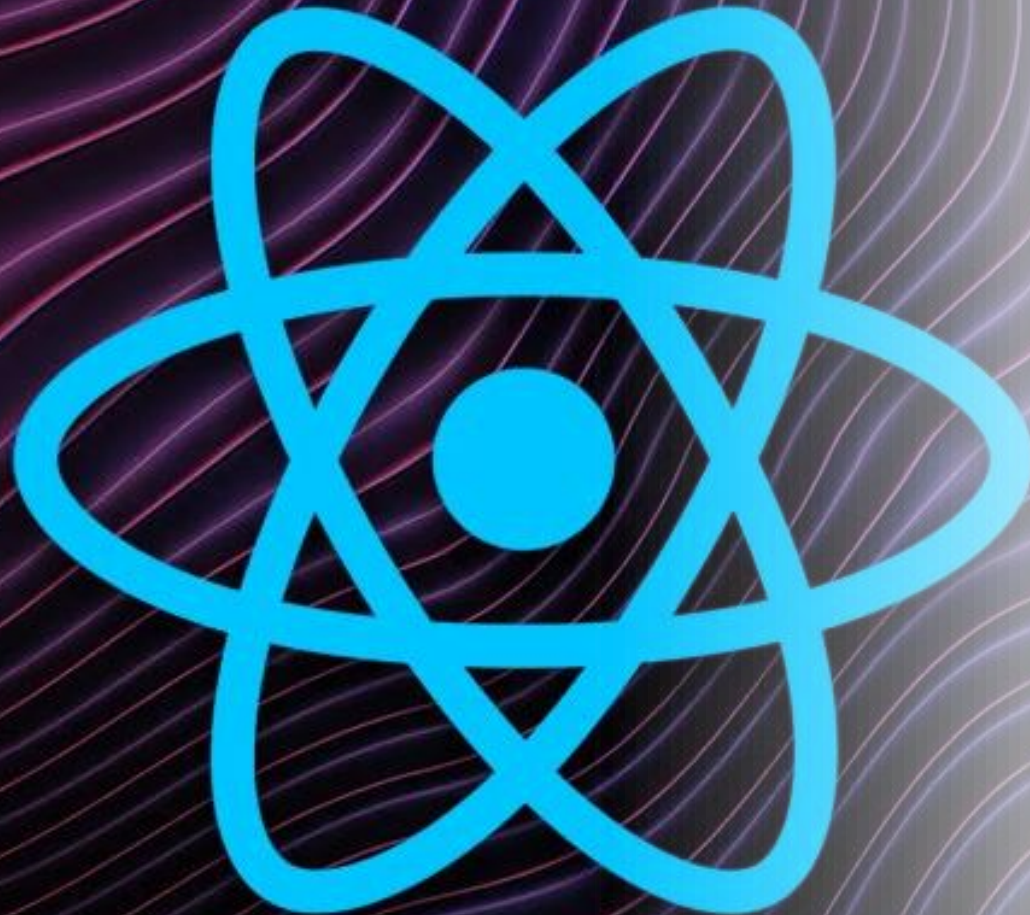




Asztali és mobil alkalmazások fejlesztése és tesztelése tantárgy – Mobil alkalmazások rész

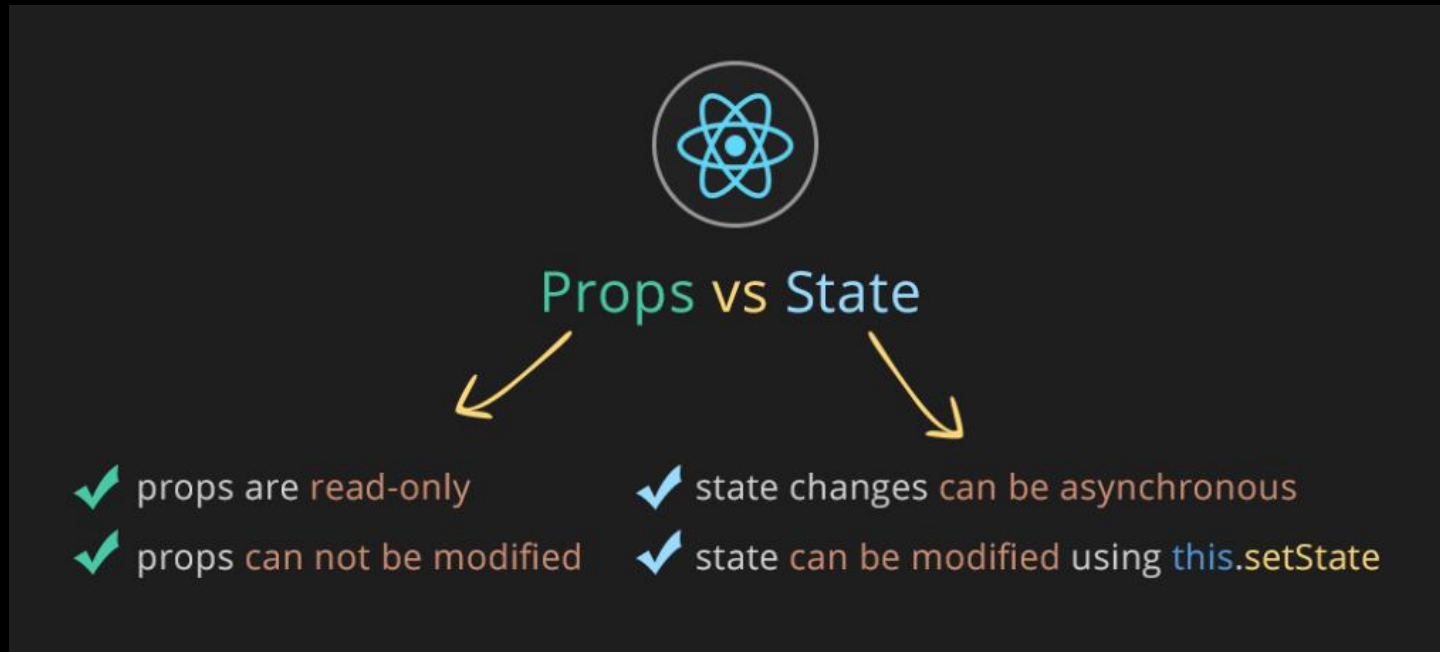
13. A osztály

Tanárok: Dr. Koncz Adrienn és
Simon Martin

2025.10.02.

State és Props

- A State és a Props két alapvető fogalom a React Native-ban (és a React-ben is), amelyek segítségével az alkalmazások logikája és felhasználói felülete irányítható és dinamikusan frissíthető.
- Mindkettő arra szolgál, hogy adatokat tároljon és megjelenítsen egy komponensben, de különböző módon működnek.



State (Állapot)

- A State egy komponens belső állapotát reprezentálja, és azt az adatot tárolja, amely idővel változhat, és amely befolyásolja a komponens megjelenését és működését.
- A State változásai újra renderelik a komponenst, így az alkalmazás dinamikusan frissülhet.

Főbb jellemzői:

- A State az egyes komponensek saját, belső állapota, amelyet a komponens maga kezel és változtathat.
- Ha a State változik, a komponens automatikusan újra renderelődik, hogy tükrözze a változásokat a felhasználói felületen.
- A State-et általában akkor használjuk, ha a komponens viselkedését vagy megjelenését dinamikusan akarjuk módosítani.

Példa a State használatára

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

const Counter = () => {
  const [count, setCount] = useState(0); // Állapot (state) létrehozása

  return (
    <View>
      <Text>{count}</Text> {/* Az állapotból származó adat megjelenítése */}
      <Button title="Növelés" onPress={() => setCount(count + 1)} /> {/* Állapot frissítése */}
    </View>
  );
};

export default Counter;
```

- Ebben a példában a count változó a komponens state-je, amely a gomb megnyomásakor növekszik, és újra rendereli a komponenst az új értékkel.

Fontos:

- A State-et csak osztály- vagy funkcionális komponensekben lehet használni, ahol a React Hook-okat (pl. useState) alkalmazzuk.
- A State módosítása aszinkron módon történik, és mindig a React-en keresztül kell végrehajtani, hogy a frissítés automatikusan történjen.

Properties – Props (Tulajdonságok)

- A Props (properties) olyan adatok, amelyeket a szülő komponens ad át a gyerek komponenseknek. A Props segítségével dinamikusan adhatunk át értékeket, és ezzel módosíthatjuk a gyerek komponensek viselkedését vagy megjelenését.

Főbb jellemzői:

- A Props csak olvasható, tehát a gyerek komponens nem módosíthatja azokat.
- A Props-ot kívülről, a szülő komponens adja át, és a komponens konfigurációjaként működik.
- A komponensek a Props segítségével testre szabhatók anélkül, hogy magukat a komponenseket módosítani kellene.

```
import React from 'react';
import {Text, View, StyleSheet} from 'react-native';

const styles = StyleSheet.create({
  center: {
    alignItems: 'center',
  },
});

const Greeting = props => {
  return (
    <View style={styles.center}>
      <Text>Hello {props.name}!</Text>
    </View>
  );
};

const LotsOfGreetings = () => {
  return (
    <View style={[styles.center, {top: 50}]}>
      <Greeting name="Rexxar" />
      <Greeting name="Jaina" />
      <Greeting name="Valeera" />
    </View>
  );
};

export default LotsOfGreetings;
```

Példa a Props használatára

- A Greeting komponens egy szöveget jelenít meg, amelyben a props.name helyére a szülő komponens által átadott érték kerül.

Fontos:

- A Props dinamikusan változtathatja a komponens megjelenését vagy működését, de maga a komponens nem tudja módosítani a Props értékeit.
- A Props-ot gyakran használják az adatok továbbítására a szülő komponensből a gyerek komponensek felé.

```

import React, {useState, useEffect} from 'react';
import {Text, View} from 'react-native';

const Blink = props => {
  const [isShowingText, setIsShowingText] = useState(true);

  useEffect(() => {
    const toggle = setInterval(() => {
      setIsShowingText(!isShowingText);
    }, 1000);

    return () => clearInterval(toggle);
  });

  if (!isShowingText) {
    return null;
  }

  return <Text>{props.text}</Text>;
};

const BlinkApp = () => {
  return (
    <View style={{marginTop: 50}}>
      <Blink text="I love to blink" />
      <Blink text="Yes blinking is so great" />
      <Blink text="Why did they ever take this out of HTML" />
      <Blink text="Look at me look at me look at me" />
    </View>
  );
};

export default BlinkApp;

```

Példa a State és Props együttes használatára

Feladatok

1. feladat: Nemzetközi Kávénap - Szöveges képernyő készítése

- Készíts egy görgethető szöveges oldalt. A feladathoz tartozó szöveget a Moodle-ben találjátok.

Kötelező elemek

- **Alap komponensek használata:**
 - ScrollView a görgethetőséghez
 - View a tartalom rendezéséhez
 - Text a címsorok és bekezdések megjelenítéséhez
- **State kezelés:**
 - Használj useState hookot a szövegek tárolásához (címek, szövegek).
- **Stílus:**
 - Legyen külön stílus a **címsorokra** és a **törzsszövegre**.
 - Adj a háttérhez egy színt (backgroundColor).
 - A címsor legyen **nagyobb, félkövér és eltérő színű**.
- **Szöveges tartalom:**
 - 3 főcím és bevezető bekezdés.



2. Feladat – az 1. feladat alapján

Szövegek összefűzése:

- A második és a harmadik cím szövegét jelenítsd meg **összekapcsolva** (pl. "Eszpresszó & Cappuccino").
- A második és a harmadik bekezdést szintén jelenítsd meg **összefűzve**.

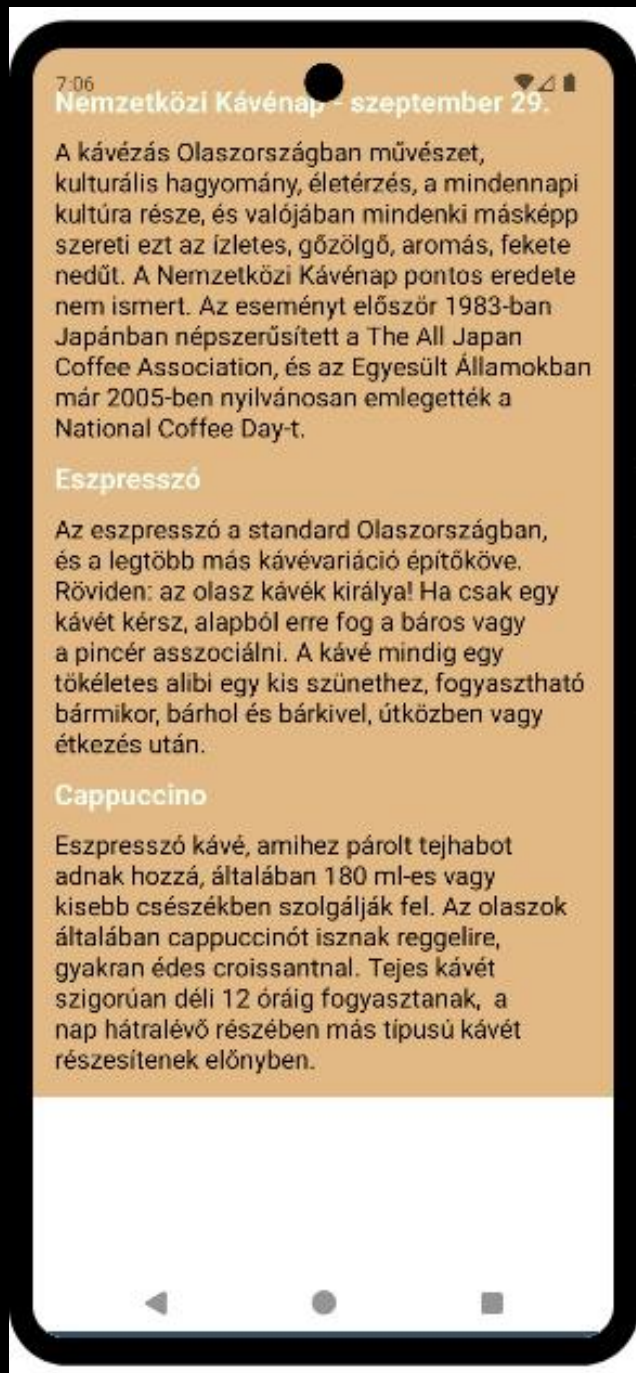


3. feladat: Külső JSON beolvasása és szövegtranszformáció

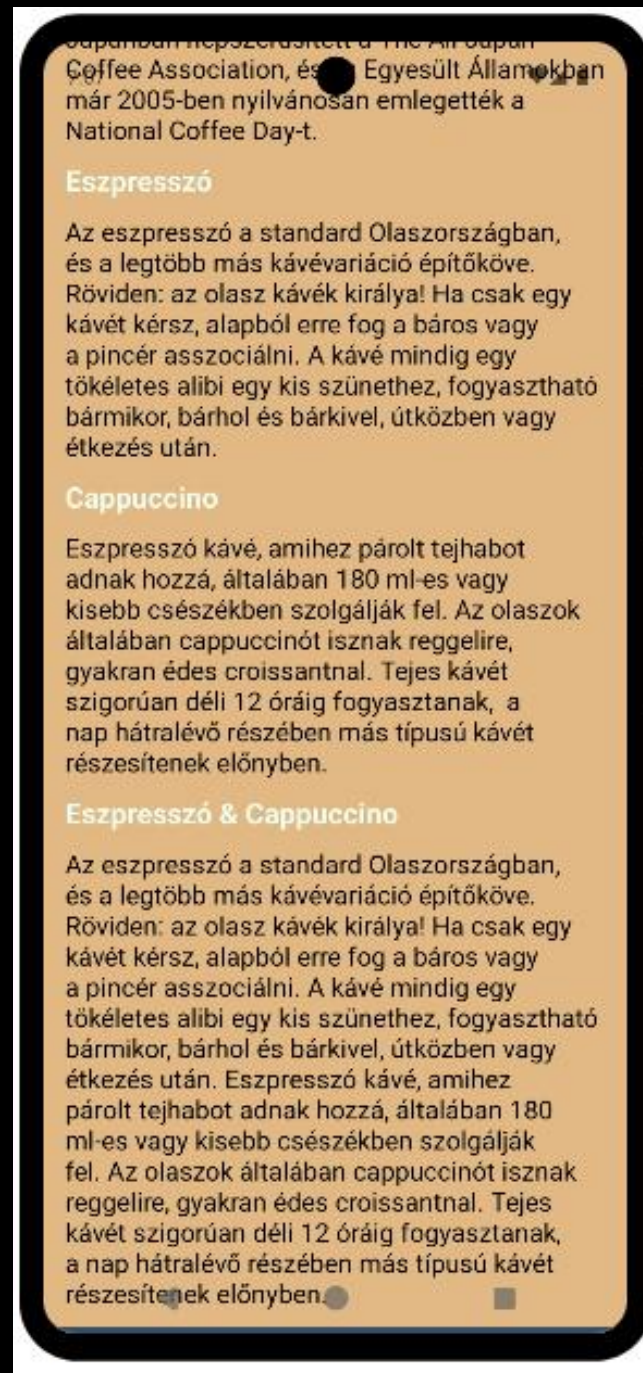
- Készítsd el úgy az első feladatot, hogy a szöveges tartalmat kiszervezed egy json fájlba (legyen a fájl neve pl. text.json)
- Függvény segítségével az első bekezdés teljes szövegét alakítsuk nagybetűsre.
- Függvény segítségével a második bekezdés szövegét csináljuk meg úgy, hogy a páratlan betűk kicsik, a páros betűk nagyok legyenek a teljes szövegben. Pl. eszpresszó = eSzPrEsSzÓ
- A harmadik bekezdés szövege szerepeljen kétszer. A második verzióban a szavak betűi legyenek random összekeverve. Ezt is függvény segítségével készítsük el.

useMemo: hook, amivel számítási eredményeket tudunk cache-elni (ne számolódjanak újra minden rendernél). A useMemo optimalizálja a teljesítményt úgy, hogy az erőforrás-igényes számításokat eltárolja (memoizálja), és csak akkor számolja újra, ha a bemeneti adatok megváltoznak.

1. feladat



2. feladat



A KÁVÉZÁS OLASZORSZÁGBAN MŰVÉSZET, KULTURÁLIS HAGYOMÁNY, ÉLETÉRZÉS, A MINDENNAPI KULTÚRA RÉSZÉ, ÉS VALÓJÁBAN MINDENKI MÁSKÉPP SZERETI EZT AZ ÍZLETES, GÖZÖLGŐ, AROMÁS, FEKETE NEDŰT. A NEMZETKÖZI KÁVÉNAP PONTOS EREDETE NEM ISMERT. AZ ESEMÉNYT ELŐSZÖR 1983-BAN JAPÁNBAN NÉPSZERŰSÍTETT A THE ALL JAPAN COFFEE ASSOCIATION, ÉS AZ EGYESÜLT ÁLLAMOKBAN MÁR 2005-BEN NYILVÁNOSAN EMLEGETTÉK A NATIONAL COFFEE DAY-T.

Eszpresszó

aZ eSzPrEsSzÓ a sTaNdArD oLaSzOrSzÁgBaN, és a lEgTöBb mÁs kÁvÉvArláCiÓ épÍTÖKöVe. rövidEn: aZ oLaSz kÁvÉk klrÁlYa! hA cSaK eGy kÁvÉT kÉrSz, aLaPbÓl eRrE fOg a bÁrOs vAgY a plnCéR aSsZoCiÁlNi. a kÁvÉ mInDiG eGy tÖkÉlEtEs aLiBi eGy kls sZüNeThEz, fOgYaSzThAtÓ bÁrMiKoR, bÁrHoL és bÁrKiVeL, úTkÖzBeN vAgY éTkEzÉS uTáN.

Cappuccino

Eszpresszó kávé, amihez párolt tejhabot adnak hozzá, általában 180 ml-es vagy kisebb csészékben szolgálják fel. Az olaszok általában cappuccinót isznak reggelire, gyakran édes croissantnal. Tejes kávé szigorúan déli 12 óráig fogyasztanak, a nap hátralévő részében más típusú kávé részesítenek előnyben.

spseóEzszt é,vák zameih olptrá jthteaob kanad ázz,ho ltnalábaá B10 -selm vayg bebkis skbéznseéc lójoklvgás fle. zA azsloko áltlnaaábl pautcócclnp nsiazk geeirgle,r rkaygan sedé sosaratln.inc jseTe étvák ariozgúns éldi 12 ágóri gaztyfonaask a npa rlvőátéah zbrésené sám úsútp éákv snrzkíeteés lnne.eöyb

3. feladat

4. Feladat – random színgenerátor készítése

- Készítsünk egy random színgenerátort csak Text, View, Image, ScrollView komponensek és useState használatával.



ImageBackground komponens

- Az ImageBackground komponens arra szolgál, hogy háttérképet adj egy View-nak.
- Hasonló, mint a CSS background-image, de React Native-ben külön komponens.
- Szintaxisa:

```
<ImageBackground  
  source={require('./images/bg.jpg')}  
  style={{ flex: 1 }}  
  imageStyle={{ opacity: 0.3 }}  
>  
  <Text>Ez a tartalom a háttérképen van</Text>  
</ImageBackground>
```

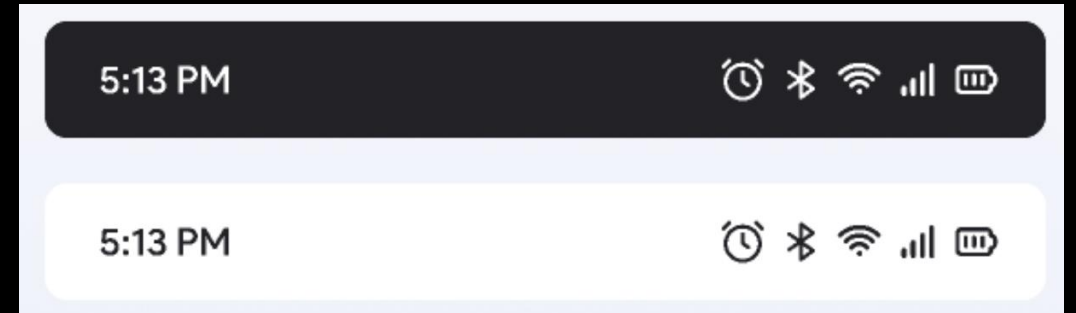
Fontos props-ok

- `source` - Kötelező. Lehet `require('./path.png')` (lokális kép) vagy `{ uri: 'https://...' }` (online kép).
- `style` - A külső konténer stílusát adja meg (pl. méret, pozíció).
- `imageStyle` - Kifejezetten a háttérkép stílusa (pl. `resizeMode`, `borderRadius`, `opacity`).
- `resizeMode` (az `imageStyle`-ben):
 - + `"cover"` – kitölti, levágja (leggyakoribb).
 - + `"contain"` – teljes képet látsz, lehet üres sáv.
 - + `"stretch"` – torzítva tölti ki.
 - + `"center"` – nem nagyít, középre teszi.

StatusBar

- A StatusBar komponens a telefon felső státuszsorát (idő, akku, jel, wifi ikonok sávja) formázza. Beállítható a szöveg színe, a háttérszíne, és az is, hogy látszódjon-e vagy rejtve legyen.
- <https://reactnative.dev/docs/statusbar>
- Szintaxisa:

```
<StatusBar  
  barStyle="light-content"  
  backgroundColor="#0f172a"  
  hidden={false}  
>
```



Fontos props-ok

- `barStyle`
 - + "default" – iOS: fekete szöveg, Android: világos vagy rendszerfüggő.
 - + "light-content" – világos ikonok, sötét háttérhez ajánlott.
 - + "dark-content" – sötét ikonok, világos háttérhez.
- `backgroundColor` (Android only) - Beállítja a sáv színét.
- `hidden` (true/false) - Elrejtí vagy mutatja a státuszbart.
- `translucent` (Android only) - Ha true, a tartalom felcsúszhat a státuszsor alá.

Stílus tippek:

- Mindig válassz olyan `barStyle`-t, ami kontrasztos a háttérrel.
- Ha a tartalom ne csússzon alá → használj `SafeAreaView`-t is.

SafeAreaView (react-native-safe-area-context)

- A SafeAreaView biztosítja, hogy a tartalom mindig a „biztonságos területen” belül maradjon → ne lógjon a notch vagy sáv alá.
- Miért kell?
 - + A modern telefonokon (pl. iPhone X és újabb) a kijelző tetején notch (kamera kivágás), alul pedig Home Indicator sáv van. Androidon is előfordulhat átlátszó státuszsor.
- Miért kell a react-native-safe-area-context csomag?
 - + A beépített SafeAreaView csak iOS-en működött rendesen.
 - + A react-native-safe-area-context csomag iOS és Android alatt is jól kezeli a safe area-t.
 - + Több funkciót ad, pl. edges prop, hook-ok (useSafeAreaInsets).

Fontos props-ok

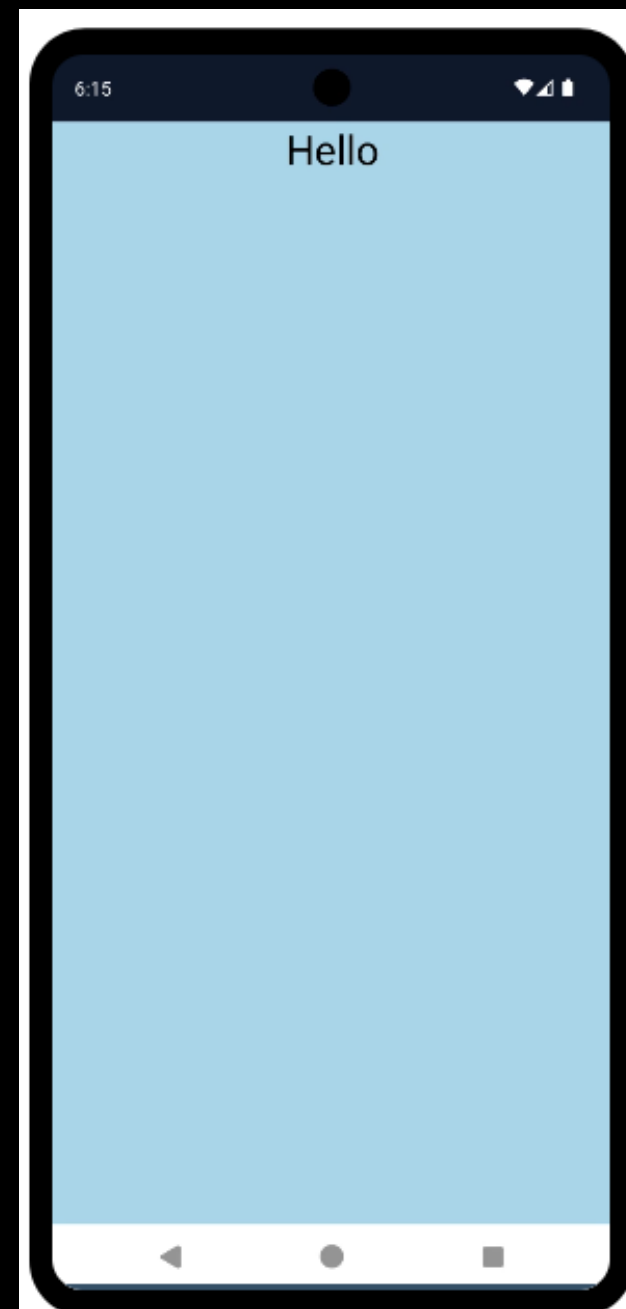
- edges: megadja, melyik oldalon legyen figyelembe véve a biztonsági sáv.
 - + Példák:['top'] – csak a státuszsor miatt.
 - + ['top', 'bottom'] – pl. iPhone home indicatorral.
 - + ['top', 'left', 'right', 'bottom'] – minden oldalon.
- style: a szokásos stílus.

```
import { SafeAreaView } from 'react-native-safe-area-context';  
  
<SafeAreaView style={{ flex: 1, backgroundColor: 'white' }} edges={['top', 'left', 'right']}>  
  <Text>Hello</Text>  
</SafeAreaView>
```

StatusBar és SafeAreaView együtt

```
import {Text, StatusBar } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';

export default function App() {
  return (
    <SafeAreaView style={{ flex: 1, backgroundColor: 'lightblue' }} edges={['top', 'left', 'right']}>
      <StatusBar barStyle="light-content"
        backgroundColor="#0f172a"
        hidden={false}
      />
      <Text style={{fontSize: 30, textAlign: 'center'}}>Hello</Text>
    </SafeAreaView>
  );
}
```



6:40

**Kovács Anna**

Középiskolás tanuló, szereti a programozást.

**Nagy Péter**

Sportkedvelő és gamer.

6:41

Állatkertünk lakói**Alpaka**

Az alpaka Dél-Amerikában élő háziasított teveléle, amelyet főként pulha gyapjáért tenyésztnek.

**Tigris**

A tigris a legnagyobb természetű macskaféle, erőteljes ragadozó, amely Ázsiában él.

**Sarki róka**

A sarki róka a tundra lakója, amely vastag bundájával jól alkalmazkodik az északi fagyos éghajlathoz.

**Jaguár**

A jaguár Dél- és Közép-Amerika legnagyobb macskaféle ragadozója, amely erős állkaposáról ismert.

**Viziló**

A viziló felig vízi életmódot.

**Krokodil**

A krokodil nagy testű hüllő.

Gyakorló feladatok

- props
- ImageBackground
- StatusBar - SafeAreaView

A panda kenguru és a leopárd folytatja emlíst, amely Afrika nagy folyókban és tavainak partvidékén él.

A leopárd nagy testű macska, amely főként folyókban és mocsarakban él és kiváló vadász.

**Panda**

A panda Kínában élő, békés medveféle, amelynek fő tápláléka bambusz.

**Ara papagáj**

Az ara papagáj színes tollú, hangutánzásáról ismert, amely főleg Amerikában él.